

CYBERSECURITY INTERNSHIP REPORT

BUILD YOUR PERSONAL CYBERSECURITY LAB

Task – 2

Building a Safe and Isolated Cybersecurity Practice Environment

Using Kali Linux, Docker, and OWASP Juice Shop

Internship Organization:

Maincrafts Technology

Submitted By:

Mehramjot Kaur

B.Tech – Computer Science & Engineering

Table of Contents

CHAPTER 1	1
Introduction.....	1
CHAPTER 2	2
Objectives	2
CHAPTER 3	4
Lab Architecture	4
CHAPTER 4	7
System Requirements and Tools Used	7
4.1. System Requirements.....	7
4.2. Software and Security Tools	7
4.3. Summary of Tools	9
CHAPTER 5	10
Lab Setup and Environment Configuration	10
5.1. Overview.....	10
5.2. Virtual Machine Configuration.....	10
5.3. Network Configuration	11
5.4. Kali Linux Verification.....	12
5.5. Docker Installation and Configuration.....	13
5.6. Deployment of OWASP Juice Shop.....	13
CHAPTER 6	15
Host Discovery, Port Scanning and Service Detection	15
6.1. Overview.....	15
6.2. Nmap Service Detection	15
6.3. Scan Results.....	15
6.4. Observations	16
6.5. Outcome	16
CHAPTER 7	17
HTTP Traffic Analysis Using Burp Suite	17
7.1. Overview.....	17

7.2. Burp Suite Configuration.....	17
7.3. HTTP Request Analysis.....	17
7.4. Observations.....	18
7.5. Outcome.....	18
CHAPTER 8.....	20
Network Packet Analysis Using Wireshark.....	20
8.1. Overview.....	20
7.2. Packet Capture Configuration.....	20
7.3. Packet Analysis.....	20
8.4. Observations.....	21
8.5. Outcome.....	21
CHAPTER 9.....	23
Risk Analysis.....	23
9.1. Overview.....	23
9.2. Identified Risks.....	23
9.3. Risk Assessment Summary.....	24
9.4. Observations.....	24
CHAPTER 10.....	25
Security Recommendations.....	25
10.1. Overview.....	25
10.2. Recommendations.....	25
10.3. Best Practices Summary.....	26
Conclusion.....	27
References.....	28

CHAPTER 1

Introduction

Cybersecurity has become an essential component of modern information technology due to the increasing number of cyber threats targeting individuals, organizations, and critical infrastructure. To effectively understand and defend against these threats, cybersecurity professionals require a controlled environment where security concepts, attack techniques, and defensive mechanisms can be studied without posing any risk to real-world systems. A cybersecurity laboratory provides such an environment by allowing security testing to be performed in a safe, isolated, and legally compliant manner.

The primary objective of a cybersecurity laboratory is to simulate realistic network and application environments for educational and research purposes. Through virtualization technologies and intentionally vulnerable applications, learners can gain practical experience in vulnerability assessment, network analysis, web application testing, and security validation. Such environments bridge the gap between theoretical concepts and practical implementation while ensuring that testing activities remain confined within an isolated infrastructure. This approach aligns with the internship task, which emphasizes creating a safe practice environment using an attacker machine, a vulnerable target, and proper network segmentation.

In this project, a personal cybersecurity laboratory was developed using Oracle VirtualBox as the virtualization platform. Kali Linux was configured as the attacker machine, while the OWASP Juice Shop application was deployed inside a Docker container as the intentionally vulnerable target application. The laboratory network was configured using NAT and Host-Only networking to provide Internet connectivity for software installation while maintaining isolation for security testing activities. This configuration follows the deployment approach described in the internship task. To validate the functionality of the laboratory, several industry-standard cybersecurity tools were utilized. Docker was used for containerized deployment of the OWASP Juice Shop application, Nmap was employed to perform host discovery and service detection, Burp Suite Community Edition was configured to intercept and inspect HTTP requests, and Wireshark was used to capture and analyze network packets exchanged between the web browser and the target application. The successful use of these tools confirmed that the laboratory environment was correctly configured and ready for future penetration testing and security assessment exercises.

The successful completion of this project resulted in a fully operational cybersecurity practice environment capable of supporting web application security testing, network traffic analysis, vulnerability validation, and future penetration testing activities. The laboratory provides a reusable foundation for advanced cybersecurity learning while ensuring that all experiments are performed within a controlled and isolated environment, minimizing risks to production systems and external networks.

CHAPTER 2

Objectives

The primary objective of this project was to design and implement a secure, isolated, and reusable cybersecurity laboratory that enables practical learning of cybersecurity concepts without affecting production systems or external networks. The laboratory was developed using virtualization and containerization technologies to simulate a real-world penetration testing environment.

The project focused on deploying Kali Linux as the attacker machine and OWASP Juice Shop as the intentionally vulnerable web application within a controlled environment. The laboratory was configured with appropriate network isolation using NAT and Host-Only networking to ensure safe experimentation while maintaining Internet connectivity for software installation and updates.

The specific objectives of this project are as follows:

- To create an isolated cybersecurity laboratory using Oracle VirtualBox.
- To deploy Kali Linux as the attacker virtual machine.
- To configure NAT and Host-Only networking for secure communication and network isolation.
- To install and configure Docker Engine for containerized application deployment.
- To deploy the OWASP Juice Shop application as the vulnerable target environment.
- To validate the deployment using network connectivity tests.
- To perform host discovery, port scanning, and service detection using Nmap.
- To capture and inspect HTTP requests using Burp Suite Community Edition.
- To analyze network traffic and packet communication using Wireshark.
- To document the complete laboratory setup, testing procedures, observations, and outcomes for future cybersecurity learning and assessment.

The successful completion of these objectives establishes a practical cybersecurity environment that can be used for web application security testing, vulnerability assessment, packet analysis, and penetration testing exercises. The laboratory also provides a scalable foundation for future internship tasks and advanced cybersecurity projects.

Outcome

After completing this chapter, the project achieved the following outcomes:

- Successfully deployed a Kali Linux attacker environment.
- Successfully hosted OWASP Juice Shop using Docker.

- Established secure network communication through NAT and Host-Only networking.
- Verified service availability using Nmap.
- Captured HTTP requests using Burp Suite.
- Analyzed network packets using Wireshark.
- Developed a reusable cybersecurity laboratory for future security testing activities.

CHAPTER 3

Lab Architecture

The cybersecurity laboratory was designed as a secure and isolated environment to support practical cybersecurity activities without affecting the host operating system or external networks. The architecture combines virtualization, containerization, and network isolation technologies to create a controlled platform for learning web application security testing and network analysis.

The laboratory was implemented on a Windows 10 host system using Oracle VirtualBox as the virtualization platform. A Kali Linux virtual machine was deployed as the attacker machine, providing a comprehensive penetration testing environment equipped with industry-standard security tools. Docker Engine was installed within the Kali Linux virtual machine to enable containerized deployment of the target application.

The OWASP Juice Shop application was deployed as a Docker container and served as the intentionally vulnerable web application. This deployment method simplified application management while ensuring consistency and portability. The application was made accessible locally through TCP port 3000, allowing interaction through a web browser and security testing tools.

To ensure both Internet connectivity and network isolation, the virtual machine was configured with two network adapters. The NAT adapter provided Internet access for downloading software packages and updates, while the Host-Only adapter created a private network that isolated the laboratory from external systems. This dual-network configuration enabled safe experimentation while preventing unintended interaction with production networks, following the architecture outlined in the internship task.

The deployed environment was validated using multiple cybersecurity tools. Nmap was used to verify host availability and identify active services, Burp Suite Community Edition was configured to intercept and analyze HTTP requests between the browser and the target application, and Wireshark was utilized to capture and examine network packets exchanged during communication. Together, these tools confirmed the successful deployment and functionality of the cybersecurity laboratory.

Components of the Lab Architecture

3.1 Host System

The host system serves as the primary operating environment and provides the computational resources required to execute the virtual machine. Windows 10 was used as the host operating system for this project.

3.2 Oracle VirtualBox

Oracle VirtualBox was used as the virtualization platform to create and manage the Kali Linux virtual machine. It provides hardware virtualization, resource allocation, and virtual networking capabilities necessary for establishing an isolated cybersecurity laboratory.

3.3 Kali Linux Virtual Machine

Kali Linux functioned as the attacker machine and contained the tools required for security testing, network analysis, and penetration testing. It also hosted Docker Engine for deploying the vulnerable web application.

3.4 Docker Engine

Docker Engine was installed inside the Kali Linux virtual machine to support containerized deployment. Using Docker ensured a lightweight, portable, and consistent environment for hosting the OWASP Juice Shop application.

3.5 OWASP Juice Shop

OWASP Juice Shop is an intentionally vulnerable web application developed for cybersecurity education. It contains multiple security weaknesses that allow learners to practice web application security testing in a controlled environment.

3.6 Security Validation Tools

The laboratory was validated using the following tools:

- **Nmap** – Used for host discovery, port scanning, and service detection.
- **Burp Suite Community Edition** – Used to intercept and inspect HTTP requests and responses.
- **Wireshark** – Used to capture and analyze network packets generated during

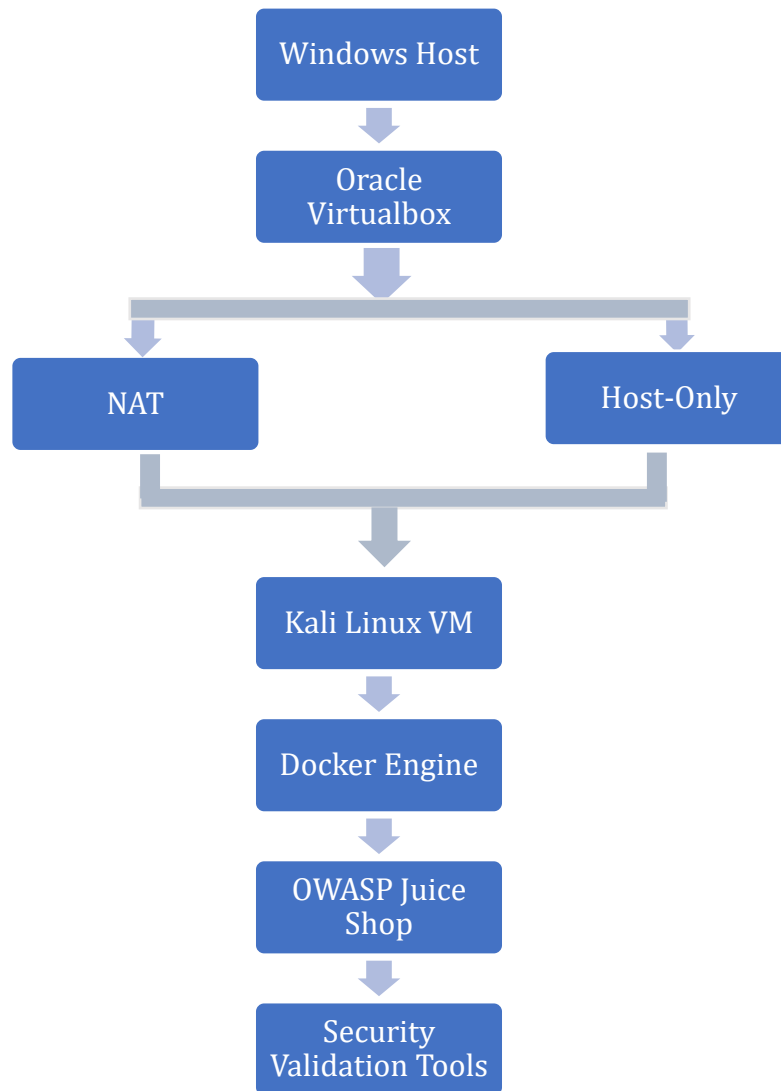


Figure 1 Architecture of the Personal Cybersecurity Lab Environment

Chapter Summary

This chapter described the overall design and implementation of the cybersecurity laboratory. It explained the role of each component, including the host system, virtualization platform, attacker virtual machine, Docker container, vulnerable web application, and security validation tools. The architecture provides a secure, isolated, and reusable environment for performing practical cybersecurity exercises and forms the foundation for the validation activities presented in the following chapters.

CHAPTER 4

System Requirements and Tools Used

4.1. System Requirements

To successfully build and execute the cybersecurity laboratory, a computer system capable of running virtualization software and multiple security tools was required. The hardware and software configuration ensured smooth deployment of the Kali Linux virtual machine, Docker Engine, and the OWASP Juice Shop application while maintaining stable performance during security testing.

The recommended system requirements for the laboratory are presented in Table 4.1.

Table 1 System Requirements

Component	Requirement
Operating System	Windows 10
Processor	Virtualization Enabled (Intel VT-x / AMD-V)
RAM	Minimum 8 GB
Free Disk Space	Minimum 40 GB
Virtualization Software	Oracle VirtualBox
Internet Connection	Required for software installation and updates

The above requirements are based on the internship task recommendations and the implementation environment used for this project.

4.2. Software and Security Tools

Several software applications and cybersecurity tools were used to build, validate, and analyze the laboratory environment. Each tool served a specific purpose during different stages of the project.

Oracle VirtualBox

Oracle VirtualBox was used as the virtualization platform to create and manage the Kali Linux virtual machine. It provided hardware virtualization, resource allocation, and network adapter configuration required to establish an isolated cybersecurity laboratory.

Kali Linux

Kali Linux served as the attacker operating system. It contains numerous penetration testing and digital forensics tools that support security assessment, vulnerability analysis, and network testing.

Docker Engine

Docker Engine was installed inside the Kali Linux virtual machine to deploy the OWASP Juice Shop application as a lightweight container. Containerization simplified application deployment while ensuring consistency across different environments.

OWASP Juice Shop

OWASP Juice Shop was deployed as the intentionally vulnerable target web application. It provides multiple built-in security vulnerabilities that allow learners to safely practice web application security testing techniques.

Mozilla Firefox

Mozilla Firefox was used to access the OWASP Juice Shop application and generate web traffic during testing. It also served as the browser configured to communicate with Burp Suite during HTTP request analysis.

Nmap

Nmap (Network Mapper) was used to perform host discovery, port scanning, and service detection. It verified that the OWASP Juice Shop application was running successfully and accessible through TCP port 3000.

Burp Suite Community Edition

Burp Suite Community Edition was used as an intercepting proxy to capture, inspect, and analyze HTTP requests and responses exchanged between the web browser and the target application.

Wireshark

Wireshark was used to capture and analyze network packets generated during communication with the OWASP Juice Shop application. Packet analysis was performed to observe TCP and HTTP communication within the isolated laboratory.

4.3. Summary of Tools

Table 2 Software and Security Tools

Tool	Purpose
Oracle VirtualBox	Virtualization Platform
Kali Linux	Attacker Virtual Machine
Docker Engine	Container Platform
OWASP Juice Shop	Vulnerable Target Application
Mozilla Firefox	Web Browser
Nmap	Port Scanning and Service Detection
Burp Suite Community Edition	HTTP Request and Response Analysis
Wireshark	Packet Capture and Network Analysis

Chapter Summary

This chapter presented the hardware requirements and software tools used to build the cybersecurity laboratory. Each tool played a specific role in establishing, validating, and analyzing the isolated environment. Together, these technologies enabled the successful deployment of a practical cybersecurity lab suitable for web application security testing and network analysis.

CHAPTER 5

Lab Setup and Environment Configuration

5.1. Overview

The cybersecurity laboratory was implemented using virtualization and containerization technologies to create a secure and isolated environment for practical security testing. The implementation process involved configuring Oracle VirtualBox, deploying the Kali Linux virtual machine, setting up network adapters, installing Docker Engine, and deploying the OWASP Juice Shop application. After deployment, the environment was validated to ensure all components functioned correctly before performing security analysis.

5.2. Virtual Machine Configuration

Oracle VirtualBox was selected as the virtualization platform due to its reliability, ease of configuration, and support for multiple virtual networking modes. A Kali Linux virtual machine was imported and configured according to the recommended hardware specifications.

The virtual machine was allocated appropriate computing resources to ensure smooth execution of security tools and Docker containers. Network connectivity was established by configuring two virtual network adapters to provide Internet access while maintaining laboratory isolation.

Configuration Summary

Parameter	Configuration
Virtualization Platform	Oracle VirtualBox
Guest Operating System	Kali Linux
Processor Allocation	2 Virtual CPUs
Memory Allocation	4 GB RAM
Network Adapter 1	NAT
Network Adapter 2	Host-Only

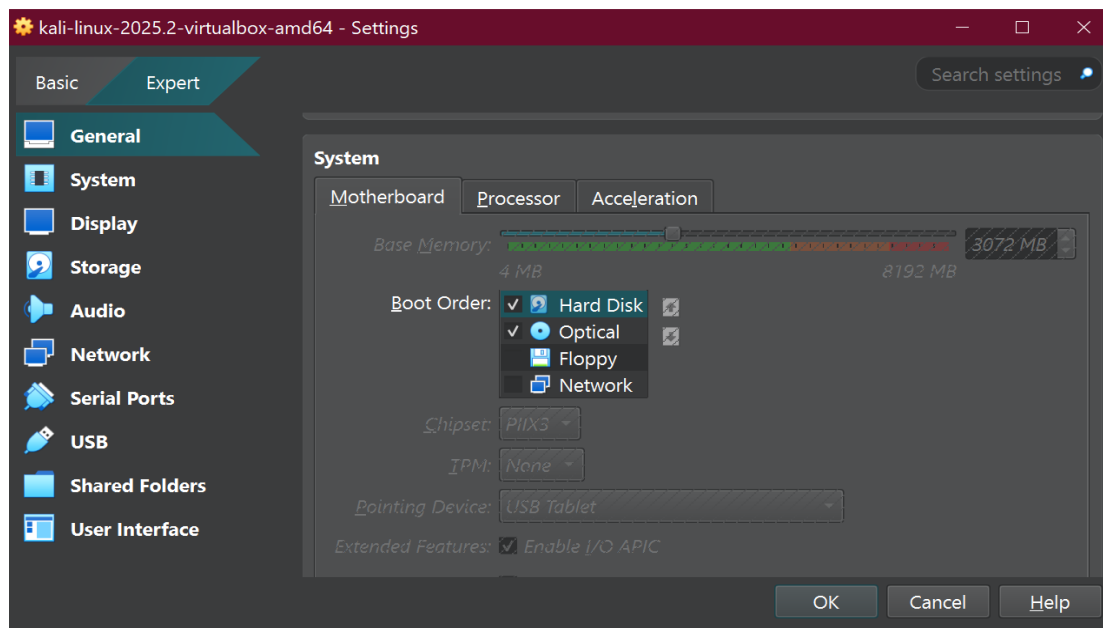


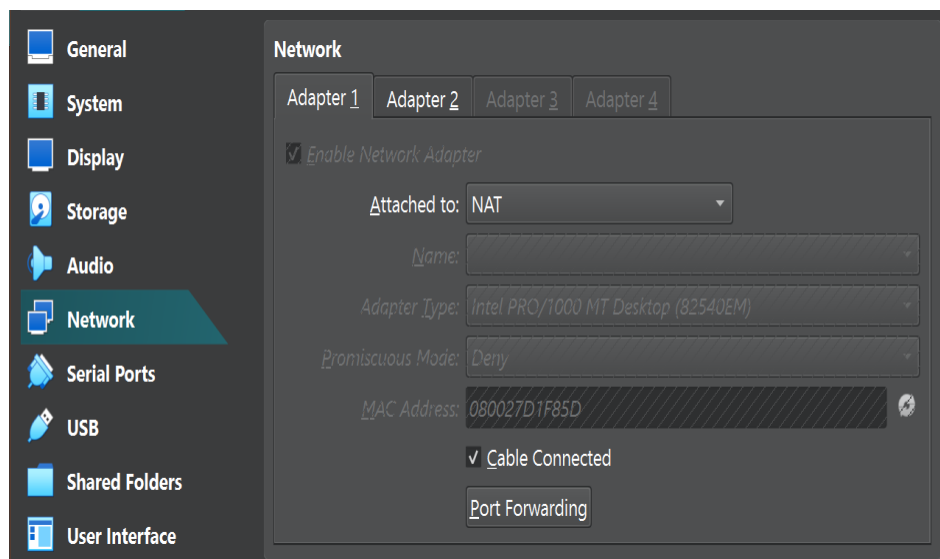
Figure 2 Oracle VirtualBox Configuration

5.3. Network Configuration

To establish a safe and isolated testing environment, two virtual network adapters were configured within the Kali Linux virtual machine.

The **NAT (Network Address Translation)** adapter provided Internet connectivity for downloading packages, updating the operating system, and installing required software. The **Host-Only Adapter** created a private network between the host machine and the virtual machine, preventing unintended exposure of the laboratory to external networks.

This dual-network configuration ensured that security testing activities remained isolated while still allowing access to online repositories for software installation.



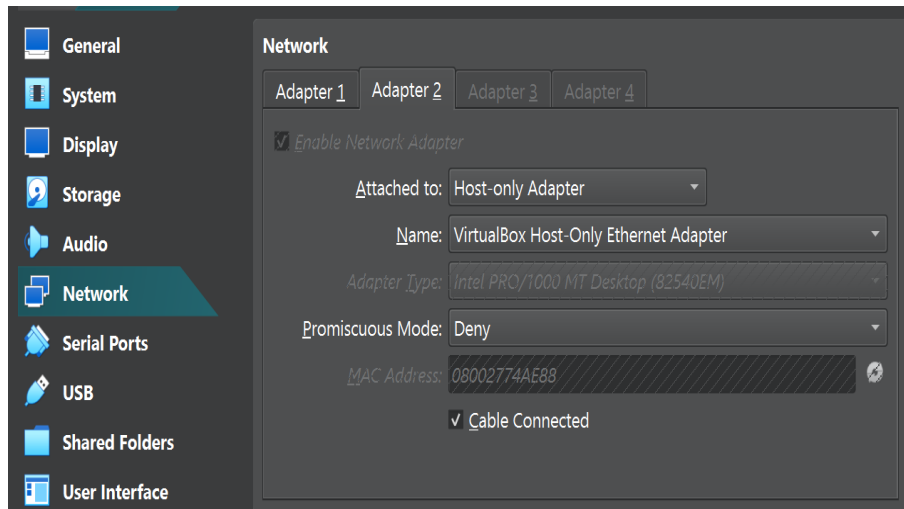


Figure 3 Network Adapter Configuration

5.4. Kali Linux Verification

After the virtual machine was started, the operating system was verified to ensure proper functionality. Network interfaces were examined using the `ip a` command, confirming successful configuration of the virtual network adapters. Internet connectivity was validated by sending ICMP echo requests to a public host.

Commands Used

`ip a`

`ping -c 4 google.com`

The successful responses confirmed that the virtual machine was correctly connected to the network and ready for software installation.

```
(kali@kali)~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 74052sec preferred_lft 74052sec
    inet6 fd17:625c:f037:2:32a2:2cd7:47d7:ff98/64 scope global dynamic noprefixroute
        valid_lft 85977sec preferred_lft 13977sec
    inet6 fe80::6f15:aee:9143:cc7d/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:74:ae:88 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 552sec preferred_lft 552sec
    inet6 fe80::90c9:346f:5a4c:a31/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether ae:e9:c5:4b:29:af brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::ace9:c5ff:fe4b:29af/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
5: veth0b92e77a1f2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
    link/ether d2:23:cc:ae:27:f9 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet6 fe80::d023:ccff:fe4e:27f9/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Figure 4 IP Configuration

```
(kali@kali)-[~]
$ ping -c 4 google.com
PING google.com (192.178.177.139) 56(84) bytes of data:
64 bytes from lcdclw-in-f139.1e100.net (192.178.177.139): icmp_seq=1 ttl=255 time=20.5 ms
64 bytes from lcdclw-in-f139.1e100.net (192.178.177.139): icmp_seq=2 ttl=255 time=16.6 ms
64 bytes from lcdclw-in-f139.1e100.net (192.178.177.139): icmp_seq=3 ttl=255 time=17.6 ms
64 bytes from lcdclw-in-f139.1e100.net (192.178.177.139): icmp_seq=4 ttl=255 time=18.3 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 16.554/18.254/20.527/1.458 ms
```

Figure 5 Internet Connectivity Verification

5.5. Docker Installation and Configuration

Docker Engine was installed within the Kali Linux virtual machine to provide a lightweight and efficient platform for deploying containerized applications. Following installation, the Docker service was enabled and verified before downloading the OWASP Juice Shop image.

The installation process included updating the package repository, installing Docker Engine, and confirming successful operation using Docker commands.

Commands Used

```
sudo apt update
```

```
sudo apt install docker.io -y
```

```
sudo systemctl enable --now docker
```

```
docker --version
```

The successful execution of these commands confirmed that Docker Engine was correctly installed and operational.

```
(kali@kali)-[~]
$ docker --version
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4ac43aed
```

Figure 6 Docker Installation Verification

5.6. Deployment of OWASP Juice Shop

The OWASP Juice Shop application was deployed as a Docker container. The application image was downloaded from Docker Hub and executed locally, making the web application accessible through TCP port **3000**.

Commands Used

```
sudo docker pull bkimminich/juice-shop
```

```
sudo docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop
```

The running container was verified using the Docker process listing command.

```
sudo docker ps
```

```
(kali@kali)-[~]  
└─$ sudo docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop  
91904eed25729094d83c92615ea07c6a85085ba4f130573a8ae0c9f8c6f2aa7e
```

Figure 7 Docker Container Deployment

```
(kali@kali)-[~]  
└─$ sudo docker ps  
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS                               NAMES  
91904eed2572   bkimminich/juice-shop "/nodejs/bin/node /j..." 28 seconds ago Up 28 seconds 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp juice-shop
```

Figure 8 Running Docker Container

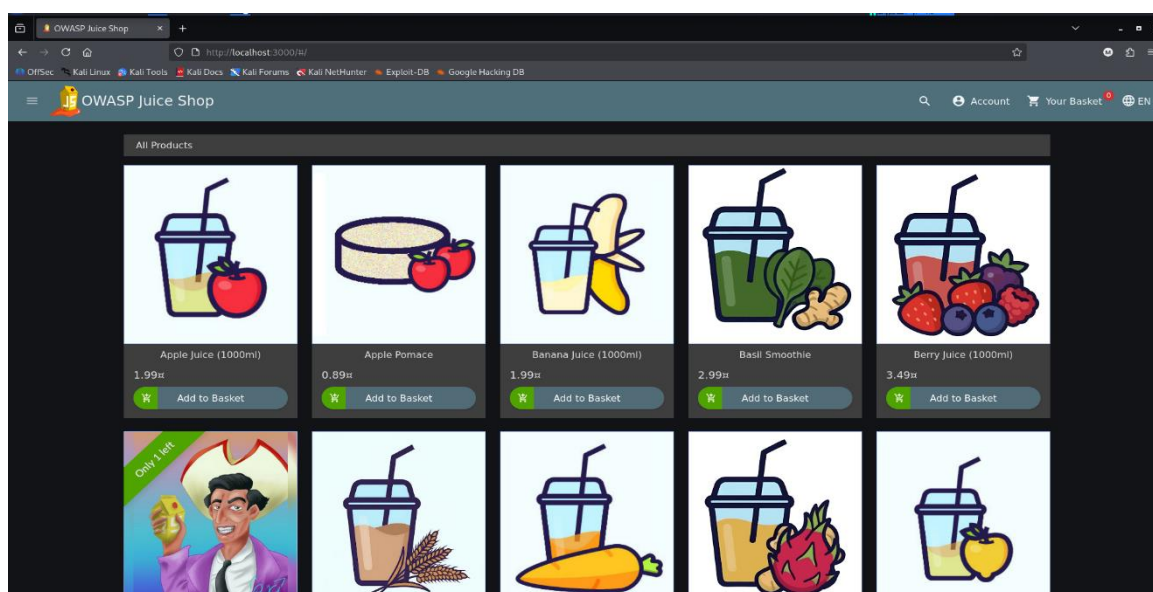


Figure 9 OWASP Juice Shop Homepage

Chapter Summary

This chapter described the implementation and configuration of the cybersecurity laboratory. The virtual environment was successfully established using Oracle VirtualBox, Kali Linux, Docker Engine, and OWASP Juice Shop. Network connectivity and software installation were verified, resulting in a fully operational and isolated laboratory ready for security validation and analysis.

CHAPTER 6

Host Discovery, Port Scanning and Service Detection

6.1. Overview

After successfully deploying the cybersecurity laboratory, the next step was to validate the availability of the target application and identify the services running on the system. This validation was performed using **Nmap (Network Mapper)**, an industry-standard network scanning tool widely used for host discovery, port scanning, and service identification.

The purpose of this activity was to verify that the OWASP Juice Shop application was successfully deployed and accessible through the expected network port before performing further security testing.

6.2. Nmap Service Detection

Nmap was executed against the locally hosted OWASP Juice Shop application to identify active network services and confirm that the application was listening on the correct port.

The following command was used during the assessment:

```
nmap -sV 127.0.0.1 -p 3000
```

Command Explanation

Parameter	Description
nmap	Starts the Network Mapper utility.
-sV	Performs service version detection on the target port.
127.0.0.1	Refers to the local host where the OWASP Juice Shop application was running.
-p 3000	Restricts the scan to TCP port 3000.

6.3. Scan Results

The Nmap scan successfully detected that TCP port **3000** was open on the local machine. The scan also confirmed that the application was responding to HTTP requests, indicating that the OWASP Juice Shop container had been deployed successfully.

The scan results verified the following:

- The host system was reachable.
- TCP port **3000** was in an **Open** state.
- The web application was actively responding to HTTP requests.
- Service detection confirmed that the deployed application was accessible and operational.

CHAPTER 7

HTTP Traffic Analysis Using Burp Suite

7.1. Overview

After verifying the availability of the OWASP Juice Shop application using Nmap, HTTP communication between the web browser and the target application was analyzed using **Burp Suite Community Edition**. Burp Suite is a widely used web application security testing tool that functions as an intercepting proxy, allowing security analysts to capture, inspect, and analyze HTTP and HTTPS requests and responses exchanged between a client and a web application.

The primary objective of this activity was to validate that the deployed application was functioning correctly and to observe the communication occurring between the browser and the OWASP Juice Shop application.

7.2. Burp Suite Configuration

Burp Suite Community Edition was launched on the Kali Linux virtual machine using the default project configuration. The proxy listener was configured with the default settings, and **Intercept** was disabled to allow traffic to pass through uninterrupted while recording all requests in the **HTTP History** tab.

Mozilla Firefox was then used to access the locally hosted OWASP Juice Shop application. Various user interactions, including browsing the home page, accessing the login page, viewing products, and searching within the application, generated HTTP requests that were automatically recorded by Burp Suite.

7.3. HTTP Request Analysis

The **HTTP History** feature successfully captured communication between the browser and the OWASP Juice Shop application. Multiple HTTP requests and responses were observed, demonstrating successful interaction with the web application.

The captured traffic included:

- HTTP GET requests for loading web pages and application resources.
- HTTP POST requests generated during user authentication.
- Requests to REST API endpoints used by the application.
- HTTP response status codes such as **200 (OK)**, **304 (Not Modified)**, and **401 (Unauthorized)**.
- Request and response headers exchanged between the client and the server.

The successful capture of these requests confirmed that Burp Suite was correctly configured and that the application was responding as expected.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response...
1	http://localhost:3000	GET	/			200	10370	HTML		OWASP Juice Shop		127.0.0.1	00:16:33 29 Ju. 8080		00:16:33 29 Ju. 8080	210	
19	http://localhost:3000	GET	/			304	392	HTML				127.0.0.1	00:17:46 29 Ju. 8080		00:17:46 29 Ju. 8080	72	
36	http://localhost:3000	GET	/assets/18n/en.json			200	35940	JSON	json			127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	1938	
37	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&... ✓			200	326	text	io/			127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	421	
38	http://localhost:3000	GET	/rest/admin/application-version			200	404	JSON				127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	1190	
39	http://localhost:3000	GET	/rest/admin/application-configuration			200	23966	JSON				127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	1874	
40	http://localhost:3000	GET	/rest/languages			200	4603	JSON				127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	2604	
41	http://localhost:3000	GET	/api/challenges/?name=Score%20Board ✓			200	862	JSON				127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	2918	
42	http://localhost:3000	GET	/api/quantities/			200	8016	JSON				127.0.0.1	00:17:19 29 Ju. 8080		00:17:19 29 Ju. 8080	2462	
44	http://localhost:3000	GET	/rest/products/search?q= ✓			200	16952	JSON				127.0.0.1	00:17:21 29 Ju. 8080		00:17:21 29 Ju. 8080	665	
45	http://localhost:3000	POST	/socket.io/?EIO=4&transport=polling&... ✓			200	215	HTML	io/			127.0.0.1	00:17:21 29 Ju. 8080		00:17:21 29 Ju. 8080	35	
47	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&... ✓			200	262	text	io/			127.0.0.1	00:17:21 29 Ju. 8080		00:17:21 29 Ju. 8080	239	
48	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	00:17:21 29 Ju. 8080		00:17:21 29 Ju. 8080	93	
49	http://localhost:3000	GET	/rest/admin/application-version			304	304					127.0.0.1	00:17:21 29 Ju. 8080		00:17:21 29 Ju. 8080	100	
50	http://localhost:3000	GET	/socket.io/?EIO=4&transport=websocket... ✓			101	129		io/			127.0.0.1	00:17:22 29 Ju. 8080		00:17:22 29 Ju. 8080	354	
51	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	00:17:22 29 Ju. 8080		00:17:22 29 Ju. 8080	4441	
52	http://localhost:3000	GET	/api/challenges/?name=Score%20Board ✓			304	305					127.0.0.1	00:17:22 29 Ju. 8080		00:17:22 29 Ju. 8080	837	
53	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	00:17:22 29 Ju. 8080		00:17:22 29 Ju. 8080	48	
54	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&... ✓			200	230	text	io/			127.0.0.1	00:17:23 29 Ju. 8080		00:17:23 29 Ju. 8080	12	
55	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	00:17:23 29 Ju. 8080		00:17:23 29 Ju. 8080	16	
67	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	00:17:30 29 Ju. 8080		00:17:30 29 Ju. 8080	10	
73	http://localhost:3000	GET	/rest/admin/application-configuration			304	306					127.0.0.1	00:18:01 29 Ju. 8080		00:18:01 29 Ju. 8080	74	
74	http://localhost:3000	GET	/rest/user/whoami			200	384	JSON				127.0.0.1	00:18:01 29 Ju. 8080		00:18:01 29 Ju. 8080	21	
75	http://localhost:3000	GET	/rest/user/whoami?fields=email ✓			200	384	JSON				127.0.0.1	00:18:01 29 Ju. 8080		00:18:01 29 Ju. 8080	21	
76	http://localhost:3000	POST	/rest/user/login			401	413	HTML				127.0.0.1	00:18:02 29 Ju. 8080		00:18:02 29 Ju. 8080	242	
77	http://localhost:3000	GET	/api/quantities/			304	306					127.0.0.1	00:18:10 29 Ju. 8080		00:18:10 29 Ju. 8080	72	
78	http://localhost:3000	GET	/rest/products/search?q= ✓			304	306					127.0.0.1	00:18:10 29 Ju. 8080		00:18:10 29 Ju. 8080	33	
94	http://localhost:3000	GET	/rest/products/22/reviews			200	593	JSON				127.0.0.1	00:18:15 29 Ju. 8080		00:18:15 29 Ju. 8080	92	
95	http://localhost:3000	GET	/rest/user/whoami?fields=email ✓			304	303					127.0.0.1	00:18:15 29 Ju. 8080		00:18:15 29 Ju. 8080	164	
96	http://localhost:3000	GET	/rest/products/22/reviews			304	304					127.0.0.1	00:18:15 29 Ju. 8080		00:18:15 29 Ju. 8080	107	
97	http://localhost:3000	GET	/rest/products/30/reviews			304	304					127.0.0.1	00:18:16 29 Ju. 8080		00:18:16 29 Ju. 8080	19	
98	http://localhost:3000	GET	/rest/products/51/reviews			200	414	JSON				127.0.0.1	00:18:22 29 Ju. 8080		00:18:22 29 Ju. 8080	41	
99	http://localhost:3000	GET	/rest/user/whoami?fields=email ✓			304	303					127.0.0.1	00:18:22 29 Ju. 8080		00:18:22 29 Ju. 8080	41	
100	http://localhost:3000	GET	/rest/products/51/reviews			304	304					127.0.0.1	00:18:22 29 Ju. 8080		00:18:22 29 Ju. 8080	33	
101	http://localhost:3000	GET	/rest/products/30/reviews			127.0.0.1						127.0.0.1	00:18:29 29 Ju. 8080		00:18:29 29 Ju. 8080	34	
102	http://localhost:3000	GET	/rest/user/whoami?fields=email ✓			127.0.0.1						127.0.0.1	00:18:29 29 Ju. 8080		00:18:29 29 Ju. 8080	35	
103	http://localhost:3000	GET	/rest/products/30/reviews			200	572	JSON				127.0.0.1	00:18:33 29 Ju. 8080		00:18:33 29 Ju. 8080	70	
104	http://localhost:3000	GET	/rest/products/30/reviews			200	572	JSON				127.0.0.1	00:18:33 29 Ju. 8080		00:18:33 29 Ju. 8080	122	
105	http://localhost:3000	GET	/rest/products/search?q= ✓			304	306					127.0.0.1	00:18:51 29 Ju. 8080		00:18:51 29 Ju. 8080	87	
106	http://localhost:3000	GET	/api/quantities/			304	306					127.0.0.1	00:18:51 29 Ju. 8080		00:18:51 29 Ju. 8080	88	
108	http://localhost:3000	GET	/rest/products/1/reviews			200	781	JSON				127.0.0.1	00:18:54 29 Ju. 8080		00:18:54 29 Ju. 8080	51	
109	http://localhost:3000	GET	/rest/user/whoami?fields=email ✓			200	384	JSON				127.0.0.1	00:18:54 29 Ju. 8080		00:18:54 29 Ju. 8080	40	
110	http://localhost:3000	GET	/rest/products/1/reviews			304	305					127.0.0.1	00:18:54 29 Ju. 8080		00:18:54 29 Ju. 8080	146	
111	http://localhost:3000	GET	/rest/products/1/reviews			304	305					127.0.0.1	00:19:00 29 Ju. 8080		00:19:00 29 Ju. 8080		
112	http://localhost:3000	GET	/api/products/170-Wed%20Jul%2029... ✓			200	643	JSON				127.0.0.1	00:19:00 29 Ju. 8080		00:19:00 29 Ju. 8080		

Figure 11 Burp Suite HTTP History Showing Captured Requests

7.4. Observations

Analysis of the captured HTTP traffic revealed that the OWASP Juice Shop application communicates extensively through REST APIs. Each user interaction generated multiple HTTP requests to retrieve resources, authenticate users, and load dynamic content.

The captured responses confirmed that the application was functioning correctly and provided valuable insight into the request–response communication model used by modern web applications. This analysis also demonstrated how Burp Suite can be used to inspect application behavior and identify potential areas for future security testing.

7.5. Outcome

The Burp Suite analysis successfully achieved the following objectives:

- Verified communication between the browser and the target application.
- Captured HTTP GET and POST requests.
- Observed REST API communication.
- Analyzed HTTP response status codes.
- Validated the proper deployment of the OWASP Juice Shop application.
- Confirmed that the laboratory environment was ready for advanced web application security testing.

Chapter Summary

This chapter demonstrated the successful use of Burp Suite Community Edition to monitor and analyze HTTP communication between Mozilla Firefox and the OWASP Juice Shop application. The captured requests and responses verified that the target application was operating correctly and highlighted the effectiveness of Burp Suite as a web application security assessment tool. The results obtained from this analysis provide a foundation for future vulnerability assessment and penetration testing activities.

CHAPTER 8

Network Packet Analysis Using Wireshark

8.1. Overview

Network packet analysis plays a vital role in cybersecurity by providing visibility into the communication occurring between networked systems and applications. It enables security professionals to monitor data transmission, identify communication patterns, troubleshoot connectivity issues, and detect potential security threats.

In this project, **Wireshark** was used to capture and analyze the network traffic generated during interaction with the OWASP Juice Shop application. The objective of this activity was to observe the communication between the Mozilla Firefox browser and the locally deployed web application, thereby validating the successful operation of the cybersecurity laboratory.

7.2. Packet Capture Configuration

Wireshark was launched on the Kali Linux virtual machine, and the **Loopback (lo)** network interface was selected for packet capture. Since the OWASP Juice Shop application was hosted locally on the same virtual machine and accessed using localhost (127.0.0.1), the communication occurred through the loopback interface rather than the external network adapters.

During packet capture, various interactions such as refreshing the application, browsing products, and accessing different pages were performed to generate network traffic.

To isolate the traffic associated with the OWASP Juice Shop application, the following display filter was applied:

```
tcp.port == 3000
```

This filter displayed only the packets exchanged through TCP port **3000**, where the application was hosted.

7.3. Packet Analysis

The captured packets confirmed successful communication between the Firefox browser and the OWASP Juice Shop application. The analysis showed multiple TCP packets exchanged over the loopback interface, along with HTTP request and response messages generated during user interaction.

The packet capture revealed the following observations:

- Communication occurred between the local IP address **127.0.0.1** and TCP port **3000**.
- HTTP **GET** requests were generated when accessing application resources.
- HTTP responses were successfully returned by the web server.
- TCP packets demonstrated the establishment and termination of communication sessions.

- Continuous packet exchange confirmed that the application was operating correctly and responding to client requests.

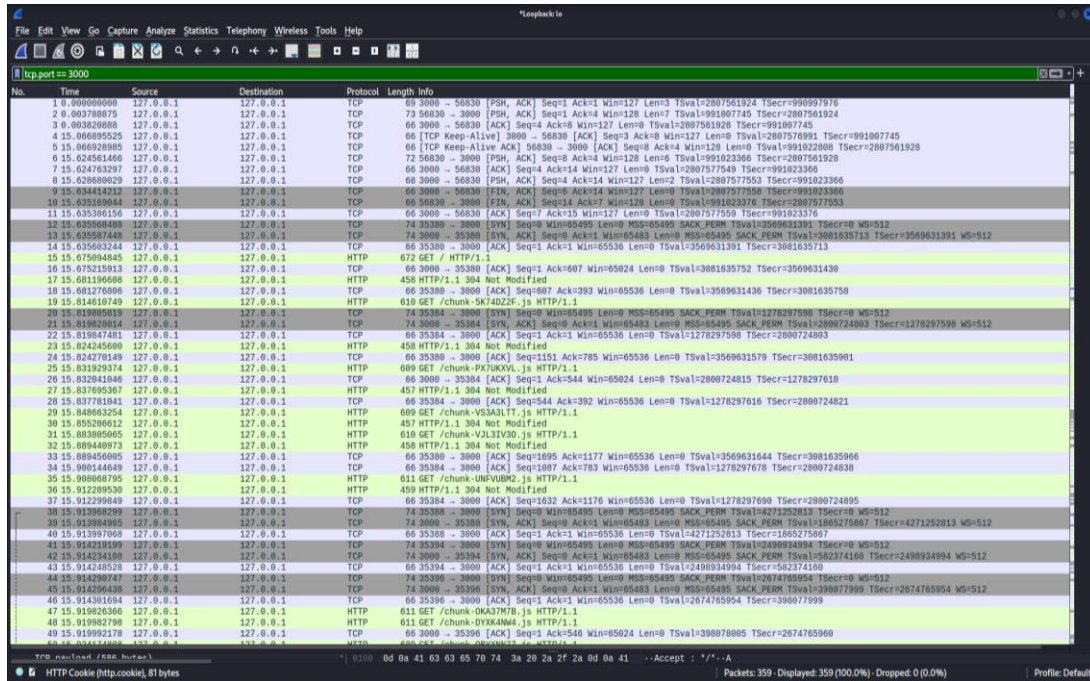


Figure 12 Wireshark Packet Capture Showing Communication on TCP Port 3000

8.4. Observations

The packet analysis confirmed that the OWASP Juice Shop application was successfully serving requests through TCP port **3000**. The use of the loopback interface ensured that all communication remained within the local system, maintaining the isolation of the cybersecurity laboratory.

The successful capture of HTTP requests and responses demonstrated that the deployed application, Docker networking, and browser communication were functioning as expected. The captured packets also illustrated how Wireshark can be used to monitor application traffic and understand the underlying communication process between a client and a web server.

8.5. Outcome

The Wireshark analysis achieved the following objectives:

- Successfully captured network traffic generated by the OWASP Juice Shop application.
- Verified communication over the loopback interface.
- Filtered traffic using tcp.port == 3000.
- Observed HTTP request and response communication.

- Validated the successful deployment and accessibility of the target application.
- Demonstrated the use of Wireshark for network traffic analysis within an isolated cybersecurity environment.

Chapter Summary

This chapter presented the use of Wireshark to capture and analyze network packets generated during interaction with the OWASP Juice Shop application. The packet analysis confirmed successful communication between the browser and the locally hosted application over TCP port 3000. The results validated the correct configuration of the cybersecurity laboratory and highlighted the importance of packet analysis in understanding application-level network communication.

CHAPTER 9

Risk Analysis

9.1. Overview

Risk analysis is an important phase of any cybersecurity assessment as it helps identify potential security concerns associated with the deployed environment and determines appropriate mitigation measures. Although the objective of this project was to build and validate a secure cybersecurity laboratory rather than perform a complete penetration test, several observations were made during the implementation and validation of the environment.

The deployed laboratory was intentionally designed to include a vulnerable web application for educational purposes. Therefore, the identified risks relate primarily to the deployment environment and emphasize the importance of maintaining proper isolation when conducting security testing.

9.2. Identified Risks

Risk 1: Deployment of an Intentionally Vulnerable Web Application

The OWASP Juice Shop application is intentionally designed with security weaknesses to facilitate cybersecurity education and penetration testing practice. If such an application were deployed in a production environment without proper safeguards, it could become a target for malicious exploitation.

Risk Level: High

Risk 2: Open Service Port

The Nmap scan confirmed that the application was accessible through TCP port **3000**. While this is required for local testing, exposing unnecessary or unprotected services on a network may increase the attack surface of a system.

Risk Level: Medium

Risk 3: Unencrypted HTTP Communication

The OWASP Juice Shop application was accessed using HTTP on the local system. HTTP traffic is transmitted without encryption and can be inspected by monitoring tools such as Burp Suite and Wireshark. While this behavior is acceptable for an isolated learning environment, production applications should use HTTPS to protect sensitive information.

Risk Level: Medium

Risk 4: Misconfigured Network Isolation

The laboratory relied on the correct configuration of NAT and Host-Only networking. Incorrect network settings could unintentionally expose the vulnerable application to external systems, increasing the likelihood of unauthorized access.

Risk Level: High

9.3. Risk Assessment Summary

Table 3 Risk Assessment

Risk	Impact	Likelihood	Risk Level
Deployment of vulnerable application	High	High	High
Open TCP port 3000	Medium	Medium	Medium
HTTP communication without encryption	Medium	Medium	Medium
Incorrect network configuration	High	Medium	High

9.4. Observations

The laboratory was successfully isolated using VirtualBox networking, ensuring that the vulnerable application remained confined to the testing environment. Network validation confirmed that communication occurred only within the intended laboratory setup, allowing security testing activities to be performed safely.

The use of Docker simplified deployment and reduced configuration complexity, while Nmap, Burp Suite, and Wireshark successfully validated the functionality of the environment. These observations indicate that the laboratory was correctly configured for educational and research purposes.

CHAPTER 10

Security Recommendations

10.1. Overview

Based on the successful implementation and validation of the personal cybersecurity laboratory, several recommendations are proposed to improve the security, reliability, and usability of similar environments. These recommendations are intended to help maintain a safe learning platform while following cybersecurity best practices.

The recommendations focus on virtualization, network isolation, system maintenance, secure configuration, and responsible use of intentionally vulnerable applications.

10.2. Recommendations

1. Maintain Proper Network Isolation

Cybersecurity laboratories should always operate within an isolated network environment. Configuring **NAT** and **Host-Only** networking prevents intentionally vulnerable applications from being exposed to external or public networks. Proper isolation minimizes the risk of unauthorized access while allowing controlled security testing.

2. Keep the Operating System and Security Tools Updated

Regular updates should be applied to the Kali Linux operating system and all installed security tools, including Docker, Nmap, Burp Suite, and Wireshark. Updated software ensures compatibility, improved performance, bug fixes, and protection against known vulnerabilities.

3. Deploy Vulnerable Applications Only in Controlled Environments

Applications such as OWASP Juice Shop are intentionally designed with security weaknesses for educational purposes. These applications should never be deployed on production servers or publicly accessible systems. Their use should remain restricted to isolated laboratory environments.

4. Use HTTPS in Real-World Deployments

Although HTTP communication is acceptable for a local testing environment, production web applications should always use **HTTPS** with valid TLS certificates. Encrypted communication protects sensitive information and reduces the risk of interception or manipulation during transmission.

5. Regularly Monitor Network Traffic

Packet analysis tools such as Wireshark should be used periodically to monitor network communication and identify unusual traffic patterns. Regular traffic monitoring helps improve visibility into application behavior and supports early detection of potential security issues.

6. Verify Running Services

Network scanning tools such as Nmap should be used to identify open ports and active services. Regular verification helps ensure that only required services are accessible, reducing the overall attack surface of the system.

7. Secure Docker Deployments

Docker containers should be managed carefully by:

- Running only trusted container images.
- Removing unused containers and images.
- Limiting unnecessary port exposure.
- Keeping Docker Engine updated.
- Monitoring container activity regularly.

These practices contribute to a more secure and manageable containerized environment.

8. Document Laboratory Activities

Maintaining detailed documentation of laboratory configurations, commands, observations, screenshots, and analysis improves reproducibility and assists future learning. Proper documentation also supports troubleshooting and project maintenance.

10.3. Best Practices Summary

Table 4 Security Best Practices

Recommendation	Purpose
Use NAT and Host-Only networking	Maintain laboratory isolation
Update Kali Linux and tools	Improve security and stability
Restrict vulnerable applications to lab environments	Prevent accidental exposure
Prefer HTTPS for production systems	Protect data in transit
Monitor network traffic	Detect abnormal communication
Perform regular Nmap scans	Identify exposed services
Secure Docker containers	Reduce container-related risks
Maintain project documentation	Improve reproducibility and maintenance

Conclusion

The objective of this project was to design, configure, and validate a personal cybersecurity laboratory for practical security learning and experimentation. The laboratory was successfully established using Oracle VirtualBox, Kali Linux, Docker Engine, and the OWASP Juice Shop application.

Throughout the project, the laboratory environment was configured with appropriate virtual networking using NAT and Host-Only adapters to ensure secure isolation. Docker was used to deploy the OWASP Juice Shop application efficiently, providing a realistic yet controlled environment for security testing.

The functionality of the laboratory was validated using multiple cybersecurity tools. Nmap confirmed that the target application was accessible through TCP port 3000 and that the required service was running correctly. Burp Suite successfully captured and analyzed HTTP requests and responses exchanged between the browser and the application, while Wireshark provided detailed packet-level analysis of network communication over the loopback interface.

The implementation demonstrated the importance of virtualization, containerization, network isolation, and traffic analysis in building an effective cybersecurity learning environment. The project also highlighted the practical application of industry-standard security tools for validating system configuration and monitoring network communication.

Overall, the cybersecurity laboratory provides a reliable platform for learning, experimentation, and future security assessments. The knowledge and practical experience gained through this project establish a strong foundation for advanced studies in ethical hacking, penetration testing, web application security, and network security.

References

- [1] OWASP Foundation, *OWASP Juice Shop Documentation*. Available: <https://owasp.org/www-project-juice-shop/>
- [2] Gordon Lyon, *Nmap Network Scanning*. Available: <https://nmap.org/>
- [3] PortSwigger Ltd., *Burp Suite Community Edition Documentation*. Available: <https://portswigger.net/burp>
- [4] Wireshark Foundation, *Wireshark User Guide*. Available: <https://www.wireshark.org/>
- [5] Docker Inc., *Docker Documentation*. Available: <https://docs.docker.com/>
- [6] Offensive Security, *Kali Linux Documentation*. Available: <https://www.kali.org/docs/>
- [7] Oracle Corporation, *Oracle VM VirtualBox User Manual*. Available: <https://www.virtualbox.org/manual/>